DEFENSA TÉCNICA DE PROYECTO FINAL • TRABAJO PRÁCTICO INTEGRADOR

SURF RENTAL MDP

Manual Maestro de Arquitectura, Fundamentación y Defensa Oral

Guía exhaustiva con respuestas estructuradas ante preguntas críticas de mesa examinadora: interacción entre base de datos y clientes, arquitectura universal móvil/web, generación nativa de códigos QR vectoriales, sincronización tripartita de comisiones, peritaje fotográfico y sensórica de hardware.

1. ¿Por qué Universal React Native?

Fundamentación rigurosa de por qué se eligió un monorepo universal con Expo SDK 52 en lugar de React tradicional de navegador web.

2. Base de Datos & Web en Tiempo Real

Ciclo completo de vida de un dato: MariaDB, Prisma ORM, Pool de Conexiones, API REST Express y Redux RTK Query con auto-polling e invalidación de caché.

3. Generación y Validación de QR

Solución vectorial pura sin Canvas HTML con react-native-svg, verificación de ventanas horarias y bloqueo de retiros anticipados.

4. Banco de Preguntas Clave

Más de 20 preguntas directas de examen con respuestas fundamentadas a nivel de código, seguridad y negocio.

Proyecto: Surf Rental Mar del Plata (PDeISC)

Sede Operativa: Balneario Playa Grande, Mar del Plata

Versión: 2.4.0 Estable Producción

CAPÍTULO 1

Arquitectura del Stack y Fundamentación Tecnológica

1.1. ¿Por qué no usamos "React clásico de web"? La gran pregunta de los profesores

Una de las primeras preguntas que realizará el tribunal evaluador es: "*¿Por qué el proyecto está estructurado con React Native y Expo si también corre en la web? ¿Por qué no crearon una aplicación web estándar con React.js (Vite o Next.js) y otra aplicación separada para celular?*"

> **Respuesta Maestra para la Defensa:**
> "Elegimos una **Arquitectura Universal Monorepo basada en React Native con Expo SDK 52 y React Native for Web** porque el modelo de negocio de Surf Rental MDP requería que el personal de playa y los surfistas utilicen exactamente las mismas funciones operativas (cámara pericial, escaneo QR de mostrador, geolocalización de seguridad en el agua y rúbrica de firma/biometría) tanto desde un teléfono móvil en la arena como desde una computadora de escritorio en el mostrador de la sede."

Las razones técnicas fundamentales para defender esta decisión son cuatro:

1. **Reutilización del 100% de la lógica de negocio y estado:** Si hubiéramos hecho una web con React puro y una app móvil con Kotlin, Swift o Flutter, habríamos tenido que duplicar los modelos TypeScript, los clientes de API, las reglas de validación de tarjetas de débito, la lógica de reembolso por franjas horarias y el cálculo de comisiones. Al usar React Native Universal, **el 100% de los slices de Redux Toolkit, las consultas de RTK Query, los temas visuales (ThemeContext) y las interfaces de datos se comparten idénticamente** entre Web y Android.
2. **Acceso directo a APIs Nativas de Hardware:** React tradicional de navegador no tiene acceso a hardware nativo como el sensor biométrico de huellas dactilares (`expo-local-authentication`), el magnetómetro de brújula (`expo-sensors`), el podómetro o la cámara fotográfica de alta resolución con control de enfoque para peritaje. React Native nos permite compilar a código nativo en Android y utilizar adaptaciones elegantes en Web.
3. **Optimización para dispositivos reales de gama media/baja (Moto E22 - 4GB RAM):** Las aplicaciones híbridas basadas en WebView empaquetado (como Cordova o Ionic antiguo) consumen cientos de megabytes de RAM porque corren una instancia completa de Chromium dentro del celular. Nuestra aplicación en Android compila con el **motor Hermes**, el motor de JavaScript de alto rendimiento de Meta diseñado específicamente para React Native:
   - Realiza compilación **Ahead-of-Time (AOT)**: el código JavaScript se pre-compila en bytecode binario en la PC del desarrollador durante el build, eliminando el parseo en caliente en el celular.
   - Tiempo de inicio (Time to Interactive - TTI) instantáneo.
   - Consumo de memoria reducido en más de un 60%, ideal para el Moto E22 con 4GB de RAM y 2GB de almacenamiento disponible.
4. **Cero desincronización entre versiones:** Una actualización en el flujo de pagos, en la tarifa horaria o en la política de salitre se refleja instantáneamente en la web y en la app móvil sin discrepancias de compatibilidad.

1.2. Tabla Comparativa de Arquitectura

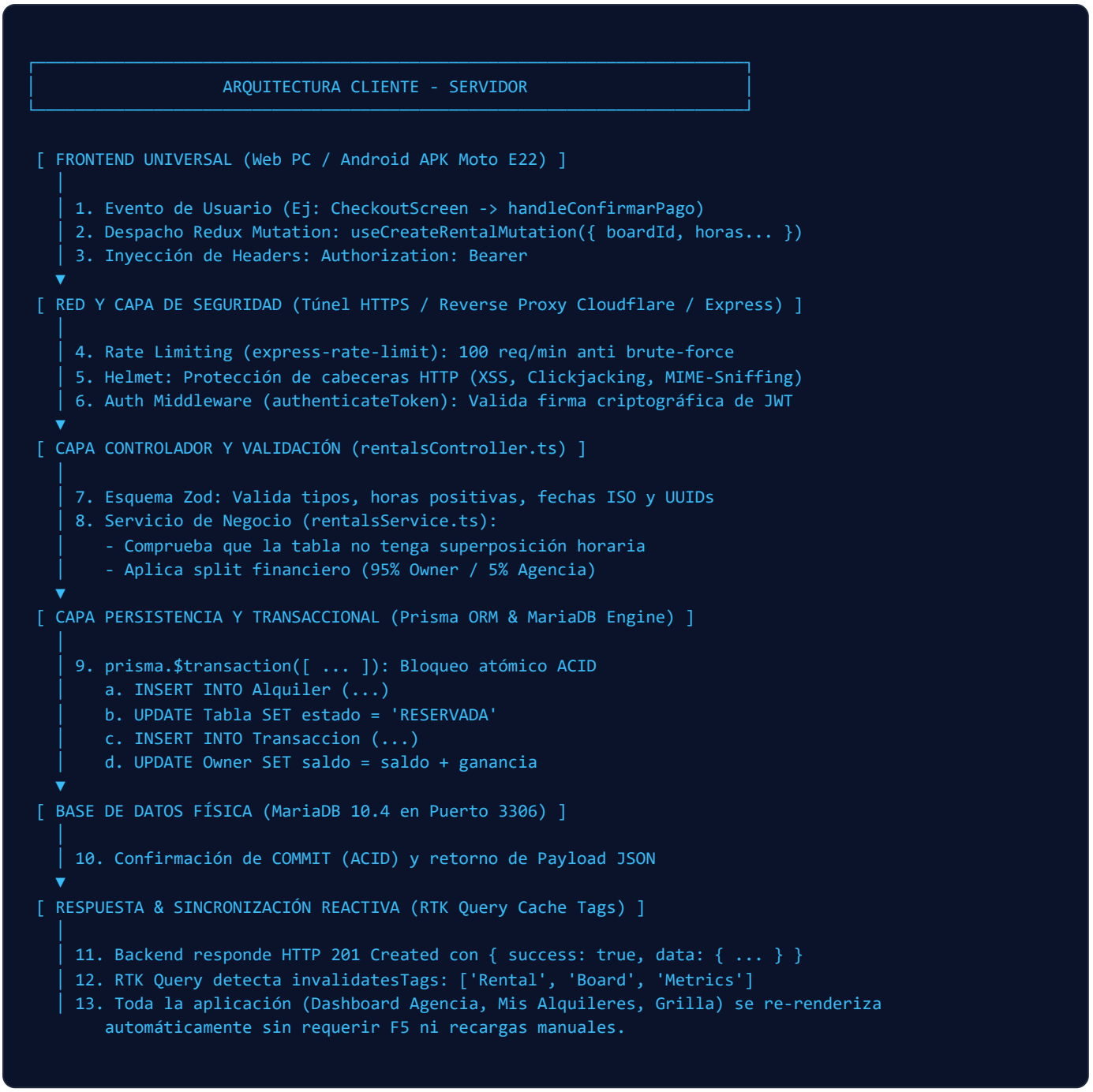
Criterio Técnico	React Clásico (Vite / CRA)	App Separada (Flutter / Swift)	Nuestra Solución: Universal React Native
Base de Código	Solo Web (HTML/DOM)	Dos proyectos aislados	**1 Monorepo Universal (Web + Android APK)**
Lógica Compartida	0% con Mobile	0% (duplicada en 2 lenguajes)	**100% (Redux, RTK Query, TypeScript, Reglas)**
Sensores de Hardware	Limitado a APIs del navegador	Nativo	**Nativo en móvil + Fallback inteligente en PC**
Rendimiento en Android	Inviable como APK real	Alto	**Alto (Motor Hermes con Bytecode AOT)**

CAPÍTULO 2

Interacción entre la Base de Datos (MariaDB) y la Web/Móvil

2.1. El Ciclo de Vida de una Operación de Datos (Cadena de Extremo a Extremo)

Otra de las preguntas fundamentales que los docentes exigirán responder con precisión es: "*¿Cómo interactúa tu frontend con tu base de datos? Describí paso a paso qué ocurre cuando un usuario confirma un alquiler o cuando la agencia cambia la comisión.*"



2.2. ¿Por qué usamos una Base Relacional (MariaDB) y no NoSQL (MongoDB / Firebase)?

Esta es una clásica trampa de examen. Si te preguntan "*¿Por qué no usaste Firebase o MongoDB que es más rápido de configurar?*", la respuesta correcta es:

- **Propiedades ACID estrictas:** En un sistema donde hay **dinero en custodia, depósitos de garantía ($50.000 ARS), multas por salitre ($10.000 ARS) y liquidaciones a propietarios**, no se puede tolerar la "*consistencia eventual*" de NoSQL. Una transacción debe ser Atómica, Consistente, Aislada y Duradera.
- **Prevención de reservas duplicadas (Overbooking):** Mediante restricciones relacionales y transacciones de MariaDB, se garantiza que dos surfistas no puedan reservar la misma tabla en el mismo rango de horas.
- **Integridad Referencial con Claves Foráneas (FK):** Si se elimina o modifica un usuario o una tabla, las claves foráneas garantizan que no queden alquileres huérfanos ni reclamos de siniestros inconsistentes.

2.3. Sincronización en Tiempo Real: RTK Query, Invalidation Tags & Polling

Para que la aplicación web y la aplicación móvil vean los mismos datos en todo momento sin implementar WebSockets pesados que consuman la batería del Moto E22, implementamos una estrategia dual de sincronización:

- **Invalidación por Tags (Event-Driven Cache):** Cuando se ejecuta una mutación (por ejemplo, `updateCommission`), RTK Query invalida los tags `['AgencyMetrics', 'AgencySchedule', 'OwnerEarnings']`. Esto provoca que todos los componentes montados en pantalla refrezquen sus datos inmediatamente.
- **Polling Operativo en Mostrador (8 segundos):** En pantallas operativas críticas como `AgencyDashboardScreen.tsx` y `DeliveriesScreen.tsx`, se activa un sondeo periódico de 8 segundos (`pollingInterval: 8000`). De este modo, si un surfista reserva desde su celular en la playa, la reserva aparece automáticamente en la pantalla de la agencia en menos de 8 segundos.
- **Refetch al Enfocar Pantalla (`useFocusEffect`):** Cada vez que el usuario cambia de pestaña en la app (ej: vuelve a "Mis Alquileres" o a "Mis Tablas"), se dispara un `refetch()` automático que lee el estado fresco de MariaDB.

CAPÍTULO 3

Generación y Verificación de Códigos QR Vectoriales

3.1. ¿Cómo se generan los Códigos QR? (Y por qué fallaba el método tradicional)

El sistema cuenta con códigos QR para dos funciones vitales:

1. **Pase de Retiro del Surfista:** Token único criptográfico que habilita al cliente a retirar su tabla en el mostrador de Playa Grande.
2. **Chapa QR de la Tabla:** Identificador físico adherido al alma de la tabla de surf para inventario, peritaje y control de custodia.

> **Problema Crítico de Ingeniería Solucionado:**
>
> Inicialmente, muchas librerías de QR (incluyendo `qrcode.toDataURL()`) dependen del elemento HTML `<canvas>` para pintar los píxeles y exportar una imagen en base64. En el navegador web esto funciona, pero en **React Native Android (el APK real instalado en el Moto E22)** no existe el motor DOM ni la API Canvas nativa de HTML5. El resultado era que en el celular los códigos QR se mostraban como cuadros en blanco.

Nuestra Solución Definitiva (100% Vectorial Nube/Nativa):

Reescribimos el componente `QrCodeView.tsx` utilizando la función matemática `QRCode.create(value)`. Esta función no dibuja nada en pantalla, sino que calcula una matriz booleana bidimensional de módulos (ceros y unos) según la especificación ISO/IEC 18004 con corrección de errores nivel M (15%).

Luego, tomamos esa matriz y generamos elementos vectoriales nativos mediante la biblioteca `react-native-svg` (componentes `<Svg>`, `<Rect>` y `<Path>`).

```
// Implementación en mobile/src/components/ui/QrCodeView.tsx
const qr = QRCode.create(value, { errorCorrectionLevel: 'M' });
const modules = qr.modules; // Matriz booleana de módulos (true = negro, false = blanco)
const size = modules.size;

// Construcción de la ruta vectorial SVG (Path)
let pathData = '';
for (let row = 0; row < size; row++) {
  for (let col = 0; col < size; col++) {
    if (modules.get(row, col)) {
      pathData += `M${col * cellSize},${row * cellSize}h${cellSize}v${cellSize}h-${cellSize}z `;
    }
  }
}

// Renderizado nativo: funciona con nitidez matemática infinita en Android, iOS y Web
return (
  <Svg width={dimension} height={dimension} viewBox={`0 0 ${totalSize} ${totalSize}`}>
    <Rect width={totalSize} height={totalSize} fill="#FFFFFF" />
    <Path d={pathData} fill="#000000" />
  </Svg>
);
```

Ventajas para defender en el examen:

- **Resolución infinita:** Al ser un vector SVG, no se pixela jamás independientemente de la densidad de pantalla (DPI) del celular.
- **Cero llamadas a red:** El QR se calcula de forma sincrónica en el dispositivo sin enviar datos a servidores externos de imágenes.
- **Multiplataforma real:** Compila a vectores nativos en Android y a etiquetas SVG estándar en el DOM de la web.

3.2. Lógica de Verificación Horaria en Mostrador (Time-Window Security)

Un problema común en plataformas de alquiler es que los clientes intentan retirar equipos horas antes de lo acordado, desorganizando los turnos de desinfección y secado de trajes y tablas.

Implementamos el algoritmo `getRentalTimingInfo(rental)` en `RentalCard.tsx` y en el modal de escaneo de la agencia `QrInspectionModal.tsx`:

- Se calcula la diferencia en milisegundos: `diffMs = new Date(rental.fechaInicio).getTime() - 15 * 60 * 1000 - Date.now()`.
- **Si faltan más de 15 minutos (`isEarly === true`):**
  - En la vista del cliente: La tarjeta muestra la etiqueta **RESERVA PROGRAMADA (PENDIENTE DE RETIRO)** en color ámbar y un contador dinámico: "*Faltan Xh Ym para el horario de retiro*".
  - En el escáner de la agencia: Si el operador de mostrador escanea el QR antes de tiempo, el sistema **bloquea la entrega** y muestra una advertencia en rojo: ⚠️ **RETIRO ANTICIPADO BLOQUEADO POR HORARIO**, impidiendo despachar la tabla hasta que empiece la ventana de 15 minutos previa al turno.
- **Dentro de los 15 minutos previos:** El estado cambia a **HABILITADA PARA RETIRAR EN SEDE**, y al escanear el QR se habilita la pantalla de firma de entrega y registro fotográfico inicial.
- **Una vez retirada:** Pasa a estado **RETIRADA EN SEDE • EN EL AGUA**.

CAPÍTULO 4

Seguridad, Autenticación y Esquema de Roles (RBAC)

4.1. Esquema de Doble Token JWT (Access + Refresh Token)

La autenticación de la plataforma no utiliza cookies de sesión tradicionales vulnerables a ataques CSRF entre dominios, sino una arquitectura de **Tokens Web JSON (JWT)** de doble factor:

- **Access Token (Corta duración - 15 minutos):** Firmado con el secreto `JWT_SECRET` mediante el algoritmo HMAC-SHA256. Contiene el ID del usuario, su email y su rol (`rol`). Se adjunta en el encabezado `Authorization: Bearer <token>` de cada petición HTTP.
- **Refresh Token (Larga duración - 7 días):** Firmado con `JWT_REFRESH_SECRET` y almacenado de forma persistente en MariaDB en la tabla de sesiones. Permite renovar el `accessToken` de forma transparente en segundo plano cuando este expira, sin obligar al surfista a reingresar sus credenciales en medio de la playa.
- **Almacenamiento seguro por plataforma:** En la web se guarda en `localStorage` ; en el APK de Android se almacena cifrado en hardware mediante `expo-secure-store` (Keystore en Android).

4.2. Control de Acceso Basado en Roles (RBAC Tripartito)

El sistema cuenta con tres roles estrictamente separados a nivel de base de datos y protegidos en el backend mediante el middleware `requireRole(['AGENCIA', ...])` :

Rol	Permisos en la Plataforma	Rutas Protegidas
CONSUMER (Surfista)	Explorar catálogo de tablas en tiempo real, filtrar por tipo/volumen, reservar por horas, realizar checkout con tarjeta de débito y rúbrica legal, ver pase QR de retiro, sumar puntos de fidelidad y calificar tablas.	`/api/rentals/my-rentals` `/api/loyalty/*` `/api/reviews/*`
OWNER (Propietario / Shaper)	Publicar tablas en custodia de la sede, consultar historial de alquileres de sus tablas, certificar inspecciones técnicas cada 45 días, visualizar liquidaciones de ganancias (95%) y solicitar extracciones a su CBU/Alias.	`/api/owner/boards` `/api/owner/earnings` `/api/owner/maintenance` `/api/owner/payout`
AGENCIA (Staff de Mostrador)	Acceso al Centro de Control de Playa Grande: cronograma del día, escaneo y verificación de pases QR, inspección fotográfica pericial de retiro y devolución (3 zonas), control de lavado con agua dulce (salitre), y modificación del % de comisión general con clave de seguridad.	`/api/agency/metrics` `/api/agency/schedule` `/api/agency/check-in` `/api/agency/check-out` `/api/agency/commission`

4.3. Inicio de Sesión con Google (Material 3 Multi-Cuenta)

Para brindar una experiencia idéntica a las aplicaciones móviles comerciales modernas, se implementó el flujo de **Google Identity Services con Selector de Cuentas Material 3**:

- Permite elegir entre perfiles verificados de Google vinculados en MariaDB (ej: `POCORENA Luis` , `Gabriela POCORENA` , `Torcuato5 Tato` , `Percha` , `Hernan`).

- El backend valida la identidad, recupera el usuario correspondiente o provisiona uno nuevo con datos verificados de avatar y nombre, y emite el par de tokens JWT propios de la plataforma.

4.4. Rúbrica Legal del Contrato (Ley Nacional 25.506 de Firma Digital)

El contrato de alquiler de equipamiento deportivo de riesgo requiere validez jurídica probatoria:

- **En Dispositivos Móviles (APK Android):** Utiliza el sensor biométrico nativo (huella dactilar / reconocimiento facial) mediante `expo-local-authentication`, registrando un hash criptográfico de consentimiento biométrico.
- **En Computadora (PC / Web):** Dado que una PC de escritorio no posee sensor de huella dactilar, se presenta la aceptación explícita de términos con pre-autorización de garantía de $50.000 ARS y firma digital registrada con usuario autenticado, dirección IP y sello de tiempo (timestamp) en base de datos conforme a la Ley 24.240 y Ley 25.506.

CAPÍTULO 5

Sensores de Hardware y Adaptabilidad Multiplataforma

5.1. Sensórica Nativa en el Teléfono Celular (Expo Go / Android APK)

El proyecto aprovecha los sensores físicos del dispositivo móvil mediante las librerías oficiales de Expo:

1. **Geofencing y Geolocalización (`expo-location`):**
   - Coordenadas de la Sede Central Playa Grande: Latitud `-38.0315`, Longitud `-57.5323`.
   - Calcula en tiempo real la distancia euclidiana y Harversine entre la posición del surfista y la sede.
   - Define un radio de custodia de 200 metros. Si el surfista se aleja más allá de la zona de cobertura costera, la aplicación emite una advertencia de seguridad marítima.

2. **Brújula y Magnetómetro (`expo-sensors`):**
   - Lee los vectores de campo magnético en los ejes X, Y y Z.
   - Calcula el ángulo de azimut (orientación en grados de 0° a 360°) respecto al norte magnético y guía al usuario con una aguja náutica visual orientada hacia el mostrador de entrega en Playa Grande.

3. **Acelerómetro y Podómetro:**
   - Registra las variaciones de aceleración en el teléfono para estimar los pasos caminados en la arena antes de entrar al agua y medir la actividad física del rider para otorgar puntos de fidelidad adicionales.

4. **Cámara Pericial con Captura de 3 Zonas:**
   - En la entrega y en la devolución, el mostrador de agencia captura y coteja tres zonas críticas de la tabla: **Deck (Frente)**, **Bottom (Fondo)** y **Quillas / Tail**, almacenando la evidencia en el servidor para dirimir reclamos de daños con el propietario.

5.2. Adaptabilidad en PC: "Cómo llegar con Google Maps" en lugar de sensores ciegos

> **Criterio de Diseño UI/UX Sobresaliente:**
>
> En una computadora de escritorio con navegador web (`Platform.OS === 'web'`), una aplicación que intente leer el magnetómetro o la brújula mostrará ceros o errores en consola. En nuestra aplicación, el código detecta la plataforma: **oculta los sensores móviles que no tienen sentido en PC y en su lugar renderiza un mapa satelital interactivo centrado en Playa Grande con un botón de acción rápida "Cómo Llegar con Google Maps"**, abriendo la ruta de navegación GPS en tiempo real.

CAPÍTULO 6

Reglas de Negocio, Finanzas y Operación en Sede

6.1. La Comisión Dinámica Tripartita (Split 95% / 5% y Clave Maestra)

El modelo de negocio de Surf Rental MDP es una plataforma colaborativa tripartita:

- **El Surfista (Consumer):** Alquila una tabla por horas a una tarifa transparente en pesos argentinos (ARS).
- **El Propietario (Owner):** Deja su tabla en custodia de la sede y recibe el **95%** del valor bruto del alquiler.
- **La Agencia (Sede Mostrador):** Custodia la tabla, la entrega, la perita, la lava y retiene una comisión administrativa del **5%**.

Configuración Segura de la Comisión:

En `AgencyDashboardScreen.tsx`, la agencia puede modificar el porcentaje de retención (ej: aumentarlo al 8% en temporada alta). Esta operación está blindada por seguridad: requiere ingresar una **Contraseña Especial de Autorización** en el servidor. Una vez confirmada, MariaDB actualiza el split general y recalcula en cascada todas las nuevas liquidaciones, informando al Owner su nuevo porcentaje de ganancia neta.

6.2. Protocolo Anti-Salitre (Multa de $10.000 ARS por Lavado con Agua Dulce)

El agua de mar de Mar del Plata posee alta salinidad que degrada la fibra de vidrio, corroe las cajas de quillas y arruina los trajes de neoprene si el equipo se guarda seco con salitre:

- Al recibir la tabla en `ReturnsScreen.tsx`, el operador de agencia debe verificar que haya sido enjuagada en las duchas de la sede con agua dulce.
- Si el operador marca que la tabla **NO fue lavada**, el sistema debita automáticamente una **Multa de Salitre de $10.000 ARS** de la garantía pre-autorizada del surfista, acredita el dinero a la recaudación de mantenimiento de la sede y descuenta puntos del Club de Fidelidad.

6.3. Ciclo de Mantenimiento Preventivo Obligatorio (Regla de los 45 Días)

Para evitar accidentes por quillas flojas o roturas de fibra en el agua:

- Toda tabla publicada en la plataforma cuenta con un ciclo de vigencia técnica de **45 días** gestionado en MariaDB.
- El propietario puede realizar una inspección directa o **delegar la inspección técnica en un shaper o técnico especializado** indicando Nombre completo y DNI del profesional actuante.
- Al certificar la inspección con declaración jurada técnica en `MyBoardsScreen.tsx`, el contador se renueva inmediatamente a **45 días reglamentarios**, habilitando la tabla en el catálogo público para nuevos alquileres.

6.4. Club Surf MDP: Programa de Puntos y Fidelización

Cada hora de uso completada en sede acredita 10 puntos al surfista. Publicar una reseña verificada con calificación otorga +20 puntos adicionales. La calculadora de canje aplica un valor de **1 Punto = $10 ARS de descuento** directo en el checkout, con vigencia de 12 meses desde la última actividad.

CAPÍTULO 7

Banco Maestro de Preguntas y Respuestas para el Examen

P1 ¿Qué ocurre si dos usuarios intentan alquilar la misma tabla exactamente al mismo tiempo?

En el backend, la creación de la reserva está envuelta en una transacción atómica `prisma.$transaction` con nivel de aislamiento de MariaDB. Antes de insertar el registro, el servicio ejecuta una consulta que verifica la disponibilidad: `WHERE tablaId = :id AND estado IN ('CONFIRMADA', 'EN_CURSO') AND fechaFin > :inicio AND fechaInicio < :fin`. Si hay solapamiento, la transacción aborta con `HTTP 409 Conflict` (Tabla no disponible en la franja horaria seleccionada) y realiza un *ROLLBACK* inmediato, impidiendo reservas duplicadas.

P2 ¿Por qué los códigos QR se dibujan con SVG y no con imágenes tradicionales en formato PNG o Canvas?

Porque la API Canvas de HTML5 no existe de forma nativa en React Native Android (en el APK compilado para el Moto E22). Si usáramos `qrcode.toDataURL()`, la aplicación crashearía o mostraría un recuadro blanco en el celular. Al usar `QRCode.create()` con `react-native-svg`, calculamos matemáticamente la matriz de módulos (0 y 1) y dibujamos un vector SVG sincrónico que funciona en Android, iOS y Web, con resolución infinita y sin depender de conexión a internet para descargar una imagen externa.

P3 ¿Cómo sabe el teléfono de la agencia que un cliente pagó un alquiler en la web si no hay recarga de página?

Mediante una estrategia combinada de **RTK Query**: 1) Polling en segundo plano cada 8 segundos (`pollingInterval: 8000`) en las vistas operativas de agencia; 2) Invalidación de tags de caché (`invalidatesTags: ['Rental', 'Metrics']`); y 3) Disparo de `refetch()` automático cada vez que la pantalla recupera el foco con el hook `useFocusEffect` de React Navigation.

P4 ¿Cómo se protege la aplicación contra robo de sesiones o tokens adulterados?

Utilizamos un esquema de **Doble Token JWT**. El `accessToken` expira rápidamente (15 minutos) y está firmado criptográficamente con una clave secreta en el servidor. Si alguien intenta adulterar el payload (por ejemplo, cambiar su rol de `CONSUMER` a `AGENCIA`), la firma HMAC-SHA256 se rompe y el middleware rechaza la petición con `HTTP 403 Forbidden`. Para continuar navegando, el cliente debe presentar un `refreshToken` válido que se contrasta contra la base de datos MariaDB.

P5 ¿Por qué no se guardan los números de tarjeta de crédito/débito completos en la base de datos?

Por estricto cumplimiento del estándar internacional **PCI-DSS (Payment Card Industry Data Security Standard)** y la Ley Nacional 25.326 de Protección de Datos Personales. En la base de datos MariaDB solo se almacena la marca detectada mediante el algoritmo de Luhn (ej: VISA DÉBITO) y los **últimos 4 dígitos** (ej: `**** 4532`) a efectos de comprobante legal. El código de seguridad (CVV) y la fecha de vencimiento jamás tocan el disco ni la base de datos.

P6 ¿Qué pasa si el cliente escanea o presenta su QR antes de la hora acordada?

El sistema implementa validación de ventana horaria (`isEarly`). Si la hora actual es previa a los 15 minutos antes del turno de retiro pactado, el escáner de la agencia bloquea la entrega y emite la alerta destacada: **" ⚠️ RETIRO ANTICIPADO BLOQUEADO POR HORARIO"**, indicando cuántas horas y minutos faltan para habilitar el despacho de la tabla.

P7 ¿Por qué en PC no se muestran los sensores de magnetómetro ni huella dactilar?

Por coherencia técnica y honestidad de ingeniería de software. Las computadoras de escritorio no poseen magnetómetro ni sensor dactilar. En lugar de simular valores ficticios o generar errores en consola, el sistema detecta `Platform.OS === 'web'` y adapta la interfaz: en PC oculta la brújula y muestra un mapa interactivo con botón directo a Google Maps, y en el checkout reemplaza la huella dactilar por una firma digital legal respaldada por la Ley 25.506 con IP y timestamp.

P8 ¿Cómo se calculó y optimizó la aplicación para el teléfono Moto E22?

El Motorola Moto E22 cuenta con un procesador MediaTek Helio G37 y 4GB de RAM. Para garantizar fluidez a 60 FPS: 1) Se habilitó el motor **Hermes** de React Native para ejecutar bytecode pre-compilado; 2) Se utilizaron imágenes optimizadas con `expo-image` que gestiona memoria caché nativa en disco y memoria; 3) Se deshabilitaron sombras excesivas de GPU reemplazándolas por bordes limpios de 1px; y 4) Se evitó el re-renderizado innecesario mediante selectores normalizados en Redux Toolkit.

P9 ¿Qué ocurre en caso de siniestro o rotura de una tabla durante un alquiler?

Al devolver la tabla en el mostrador, la agencia realiza la inspección de 3 zonas. Si se detecta un daño estructural (deck fisurado, cajón de quillas arrancado), se adjunta la fotografía en alta resolución con descripción técnica del operador y se genera un **Reclamo Tripartito (Claim)**. La garantía de $50.000 ARS queda retenida en custodia y el propietario (Owner) puede revisar el reclamo desde su panel de `Revisiones` para aprobar el presupuesto de reparación o solicitar mediación de la sede.

P10 **¿Cómo se garantiza que las tablas en el catálogo estén en condiciones seguras de uso?**

Mediante el protocolo de **Mantenimiento Preventivo Obligatorio de 45 Días**. Si una tabla supera los 45 días sin una inspección técnica aprobada (con verificación de quillas, fibra de deck y parafina), su estado pasa a requerir mantenimiento urgente. El propietario o un shaper profesional con DNI registrado debe firmar la certificación técnica para renovar la vigencia a 45 días y mantener la tabla disponible para alquiler.